Feature _ Extraction _ Techniques

1 Student Information

- Name: Afia Sultana
- ID: 22-47097-1
- Section: F
- Course Teacher: Dr. Md. Asraf Ali

2 Feature Extraction Techniques:

Feature extraction turns raw image data into useful numbers while keeping important details. These features help in tasks like object detection, classification, and image matching, making image analysis more effective.

3 Importing the libraries

```
import gdown
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import graycomatrix, graycoprops, local_binary_pattern
from skimage.feature import blob_dog
from skimage.color import rgb2gray
```

4 Importing the image

```
# Google Drive file ID
file_id = '1N5fD_iC85LUh0aQgh3Xxij5ssP55gSVg'

# Construct the URL for downloading
url = f'https://drive.google.com/uc?id={file_id}'

# Download the image
output_path = 'butterfly.jpg'
gdown.download(url, output_path, quiet=False)

# Importing the image
image = cv2.imread(output_path)
```

5 Convertion of BGR to RGB

```
# Conversion of BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

6 Visualizing the RGB image

```
# Visualizing the RGB image
plt.figure(figsize=(8, 8))
plt.imshow(image_rgb)
plt.title('RGB Image')
plt.axis('off')
plt.show()
```

RGB Image

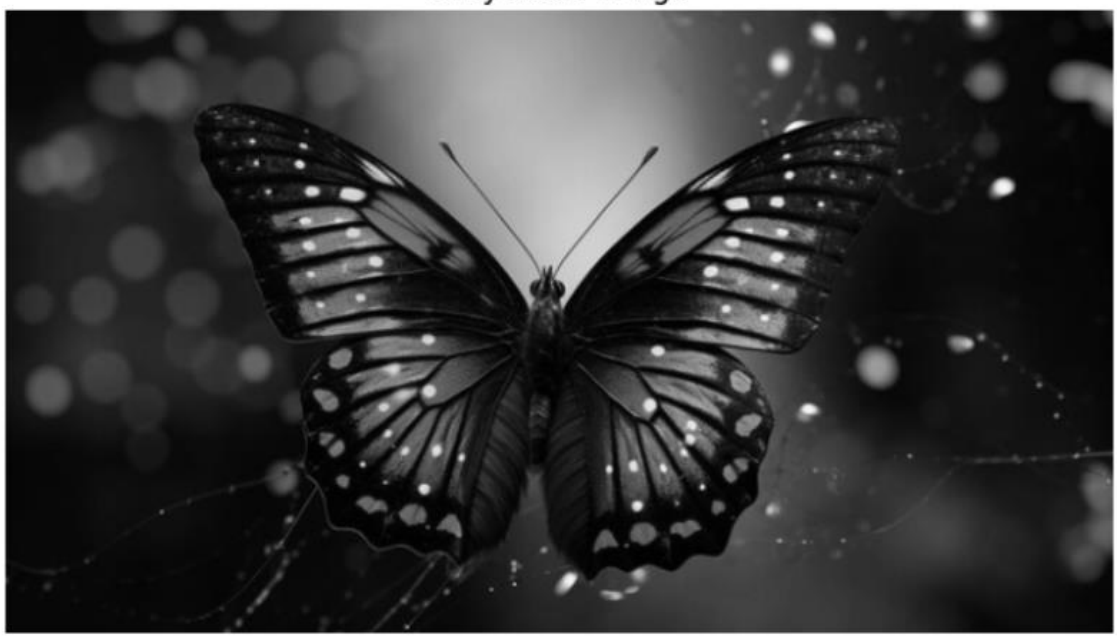
7 Converting the image into gray-scale image

```
# Converting the image into gray-scale image
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

8 Visualizing the gray-scaled image

```
# Visualizing the gray-scaled image
plt.figure(figsize=(8, 8))
plt.imshow(gray_image, cmap='gray')
plt.title('Gray-scale Image')
plt.axis('off')
plt.show()
```

Gray-scale Image



9 Color-based Features

Color-based feature extraction techniques leverage the color information within an image to derive meaningful features. These techniques are widely used in image processing, computer vision, and pattern recognition.

9.1 Calculation the Histogram for each channel

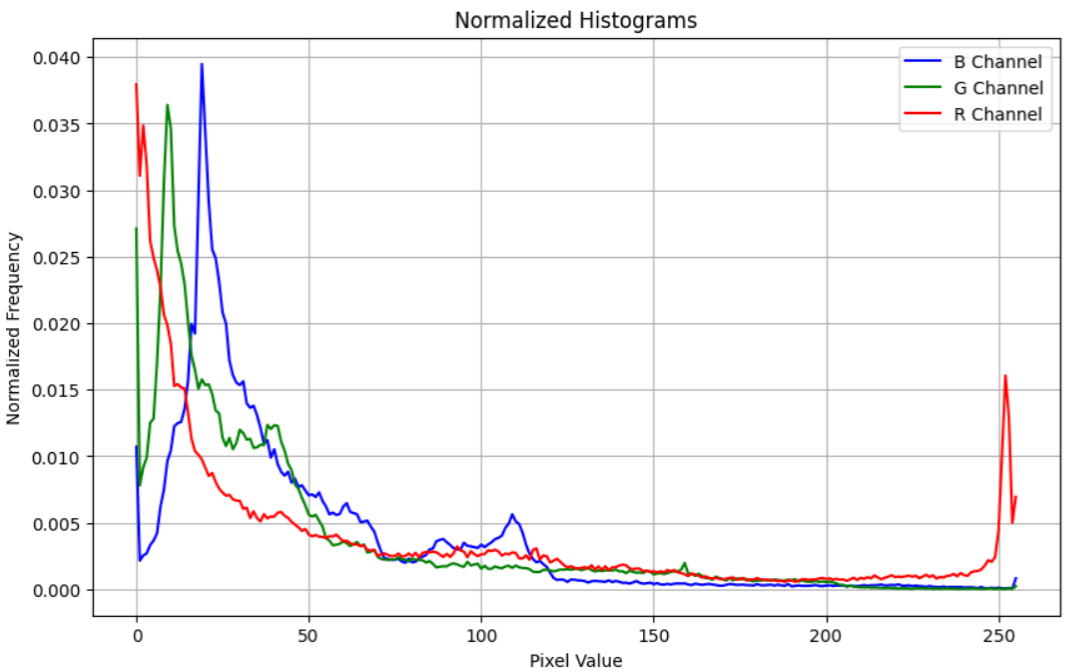
```
# Calculation of the Histogram for each channel
colors = ('b', 'g', 'r')
histograms = {}

plt.figure(figsize=(10, 6))
plt.title('Normalized Histograms')
plt.xlabel('Pixel Value')
plt.ylabel('Normalized Frequency')

for i, color in enumerate(colors):
    hist = cv2.calcHist([image], [i], None, [256], [0, 256])
    normalized_hist = hist / hist.sum()  # Normalize the histogram
    histograms[color] = normalized_hist
    plt.plot(normalized_hist, color=color, label=f'{color.upper()} Channel')
```

9.1.1 Normalizing the Histograms

Normalization makes histograms consistent by removing the effect of image size, allowing fair comparisons between images. It shows pixel intensity as probabilities, useful for tasks like image comparison and recognition.



9.1.2 Visualizing the Histograms

```
# Visualizing the Histograms
plt.legend(loc='upper right')
plt.grid()
plt.show()
```

9.2 Color Moments

Color moments are measures that can be used differentiate images based on their features of color. Once calculated, these moments provide a measurement for color similarity between images. The mean, standard deviation and skewness are commonly used color moments

- Mean: Mean can be understood as the average color value in the image

$$E_i = \frac{1}{N}\sum_{j=1}^{N} p_i \text{ j}$$

• Standard Deviation: The standard deviation is the square root of the variance of the distribution.

$$\boldsymbol{\sigma i} = \sqrt{\left(\frac{1}{N}\sum_{J=1}^{N}\left(P_{ij-E_i}\right)^2\right)}$$

• Skewness: Skewness can be understood as a measure of the degree of asymmetry in the distribution

$$si = \sqrt[3]{\left(\frac{1}{N}\sum_{j=1}^{N}(p_{ij} - E_{i)}{}^2\right)}$$

9.2.1 Calculating the mean, variance and skewness for each channel

```
from scipy.stats import skew

# Calculating the mean, variance, and skewness for each channel
channel_stats = {}
for i, color in enumerate(colors):
    channel_data = image[:, :, i].ravel()  # Flatten the channel data
    channel_mean = np.mean(channel_data)
    channel_variance = np.var(channel_data)
    channel_skewness = skew(channel_data)
    channel_stats[color] = {
        'Mean': channel_mean,
        'Variance': channel_variance,
        'Skewness': channel_skewness
    }
```

```python
# Printing the statistics
print("Channel Statistics:")
for color, stats in channel_stats.items():
    print(f"{color.upper()} Channel:")
    print(f"  Mean: {stats['Mean']}")
    print(f"  Variance: {stats['Variance']}")
    print(f"  Skewness: {stats['Skewness']}")
```

10 Texture Based Features

Texture analysis is an important part of image processing and computer vision that looks at how pixel intensities are arranged in an image. It helps in understanding the texture of an image, which is important for tasks like classifying, segmenting, and recognizing images, especially when the images have repeating patterns or complex structures.

10.1 Gray-Level Co-occurrence Matrix (GLCM)

The Gray Level Co-occurrence Matrix (GLCM) is a statistical method used to analyze the texture of an image by examining the spatial relationships between pixel intensities. Four important measures derived from the GLCM are Contrast, Correlation, Energy, and Homogeneity.

Contrast: Contrast measures the local variations in the GLCM, indicating how much the intensity of a pixel differs from its neighbors over the whole image.

$$\text{Contrast} = \sum_{i,j}(i-j)^2 \,.\, p(i,j)$$

where $P(i,j)$ is the (i,j)-the entry in the normalized GLCM.

Correlation: Correlation measures the joint probability occurrence of specified pixel pairs, de scribing how correlated a pixel is to its neighbor over the entire image.

$$\text{Correlation} = \frac{\sum_{i,j}(i-\mu_{i).(j-\mu_{j).P(i,j)}}}{6_i - 6_j}$$

where μi and μj are the means, and σi and σj are the standard deviations of the GLCM.

Energy: Energy (also known as Angular Second Moment) measures the sum of squared elements in the GLCM. It represents the textural uniformity or pixel pair repetitions in an image.

$$\text{Energy} = \sum_{i,j} p(i,j)^2$$

Homogeneity: Homogeneity (also known as Inverse Difference Moment) measures the closeness of the distribution of elements in the GLCM to the GLCM diagonal, indicating how uniform the texture is.

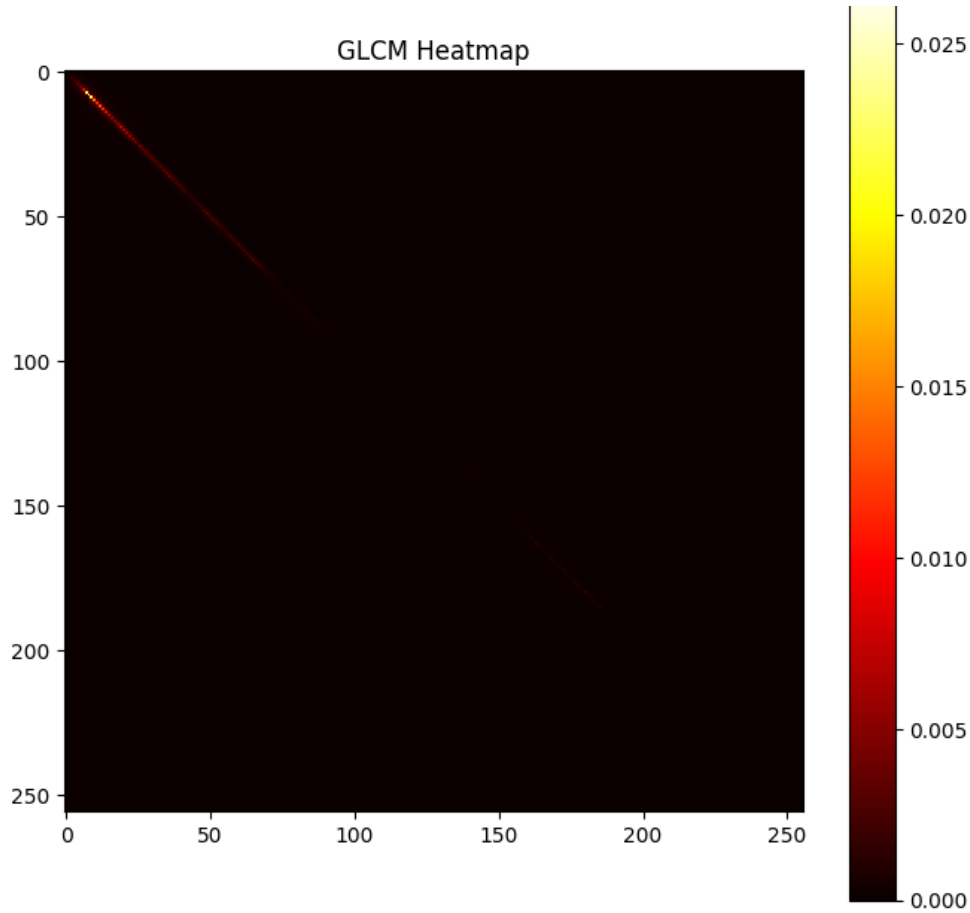
$$\text{Homogeneity} = \sum_{i,j} \frac{p(i,j)}{1+|i-j|}$$

10.1.1 Calculating the co-occurrence matrix for the image

```
# Calculating the co-occurrence matrix for the image
glcms = graycomatrix(gray_image, distances=[1], angles=[0], levels=256, symmetric=True, normed=True)
```

10.1.2 Visualizing GLCM as heatmap

```
# Visualizing GLCM as heatmap
glcms_heatmap = glcms[:, :, 0, 0]
plt.figure(figsize=(8, 8))
plt.imshow(glcms_heatmap, cmap='hot', interpolation='nearest')
plt.title('GLCM Heatmap')
plt.colorbar()
plt.show()
```




```
Texture Features:
Contrast: 105.96625065002603
Dissimilarity: 4.020662159819726
Homogeneity: 0.5047520346058338
Energy: 0.05887281069553784
Correlation: 0.9790733866118707
```

10.1.3 Calculating the texture features from the co-occurrence matrix

```
# Calculating the texture features from the co-occurrence matrix
features = {
    'Contrast': graycoprops(glcms, 'contrast')[0, 0],
    'Dissimilarity': graycoprops(glcms, 'dissimilarity')[0, 0],
    'Homogeneity': graycoprops(glcms, 'homogeneity')[0, 0],
    'Energy': graycoprops(glcms, 'energy')[0, 0],
    'Correlation': graycoprops(glcms, 'correlation')[0, 0]
}
```

10.1.4 Printing the texture features

```
# Printing the texture features
print("Texture Features:")
for feature, value in features.items():
    print(f"{feature}: {value}")
```

11 Local Binary Patterns (LBP)

LBP is a simple yet effective method for texture description. It compares the intensity of a central pixel with its neighbors, assigning a binary code based on whether each neighbor is brighter or darker. This process captures texture details and is robust to lighting changes, making it efficient for real-time applications like face recognition, texture analysis, and object detection.

1. Convert Image to Grayscale

• Convert the input image to grayscale to simplify processing:

$I(x,y)$ = 0.299 · $R(x,y)$ +0.587 · $G(x,y)$ +0.114· $B(x,y)$

• Where $R(x,y)$, $G(x,y)$, and $B(x,y)$ are the red, green, and blue color channels, respectively.

2. Compute LBP

• Compute the LBP of the grayscale image using the local_binary_pattern function. The LBP is defined for each pixel in a local neighborhood. For the uniform method, the number of neighbors P and the radius R are specified.

LBP(x,y) = $\sum * i$ =0P−1bi ·2i

where: bi = {1 if $I * neighbor, i \geq I_center$ 0 otherwise

• P =8: Number of neighboring pixels.

• R=1: Radius of the circular neighborhood.

• The method='uniform' parameter specifies that uniform patterns are used, which have at most 2 bit transitions.

3. Compute Histogram

• Compute the histogram of the LBP values. This histogram represents the frequency of each LBP code.

H =histogram(LBP,bins = np.arange(0,11),range = (0,10))

• Normalize the histogram by dividing by the sum of all histogram values: $H_normalized$ = H $\sum H$

• This normalization ensures that the histogram is a probability distribution.

4. Return LBP Features

• The normalized histogram of LBP values is used as the feature vector representing the texture of the image.

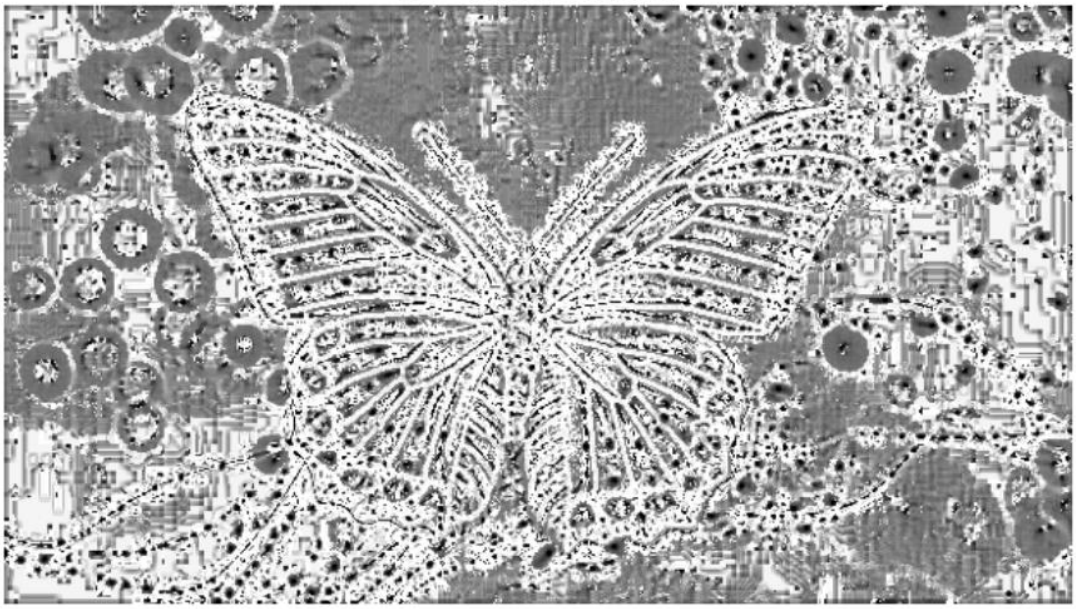
11.1 Calculating the LBP

```
# Calculating the LBP
radius = 3
n_points = 8 * radius
lbp = local_binary_pattern(gray_image, n_points, radius, method='uniform')
```

11.2 Visualizing the LBP transformation of the image

```
# Visualizing the LBP transformation of the image
plt.figure(figsize=(8, 8))
plt.imshow(lbp, cmap='gray')
plt.title('LBP Transformation')
plt.axis('off')
plt.show()
```

LBP Transformation



```
LBP Features (Histogram):
[0.0244981  0.01232693 0.01041018 0.00896071 0.00893908 0.00957079
 0.01025874 0.01436051 0.01994202 0.02840516 0.04580305 0.08438474
 0.07892004 0.07154292 0.03914417 0.03906629 0.02030547 0.0256793
 0.01308844 0.01235722 0.01318363 0.01512634 0.01838006 0.01948771
 0.08898408 0.26687435]
```

11.3 Calculating the Histogram of LBP features

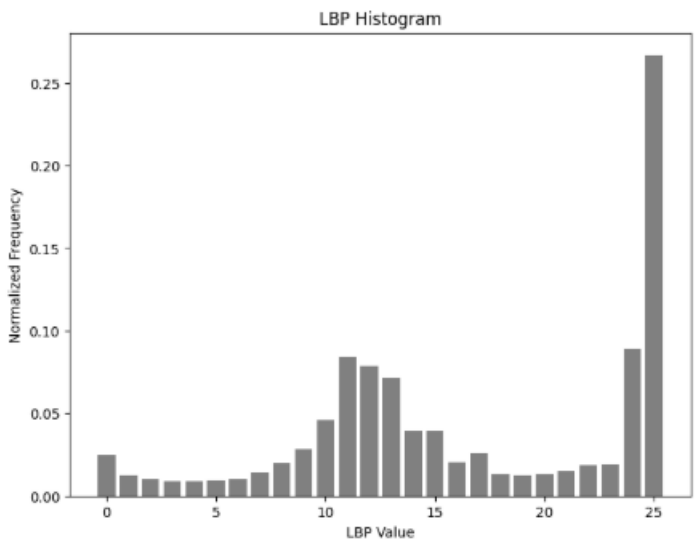
```
# Calculating the Histogram of LBP features
lbp_hist, lbp_bins = np.histogram(lbp.ravel(), bins=np.arange(0, n_points + 3), range=(0, n_points + 2))
lbp_hist = lbp_hist.astype(float) / lbp_hist.sum()
```

11.4 Printing the calculated LBP features

```
# Printing the texture features
print("Texture Features:")
for feature, value in features.items():
    print(f"{feature}: {value}")
```

11.5 Visualizing the LBP Histogram

```
# Visualizing the LBP Histogram
plt.figure(figsize=(8, 6))
plt.bar(lbp_bins[:-1], lbp_hist, width=0.8, color='gray', align='center')
plt.title('LBP Histogram')
plt.xlabel('LBP Value')
plt.ylabel('Normalized Frequency')
plt.show()
```



12 Canny Edge Detection

Canny edge detection involves smoothing, gradient calculation, non-maximum suppression, double

thresholding, and edge tracking by hysteresis to detect edges in an image while minimizing noise and false edges.

1. Gaussian Smoothing

• Smooth the image to reduce noise using a Gaussian filter:

$x = \partial I\ \partial x\ Gy = \partial I\ \partial$y

Calculate the gradient magnitude $ G$ and direction θ: $G = \sqrt{G2\ x + G2\ y}$,θ = arctan ($Gy/\ Gx$)

2.Non-Maximum Suppression

• Thin the edges by suppressing all non-maximum points in the gradient direction: – Compare each pixel's gradient magnitude to its neighbors along the gradient direction. Keep the pixel if it is the maximum.

3. Double Thresholding

• Apply high (TH) and low (TL) thresholds:

– Strong edges: $G > TH$

– Weak edges: $TL < G \leq TH$

– Suppress: $G \leq TL$

4. Edge Tracking by Hysteresis

• Connect weak edges to strong edges if they are connected:

– Strong edges are retained.

– Weak edges connected to strong edges are also retained, otherwise suppressed

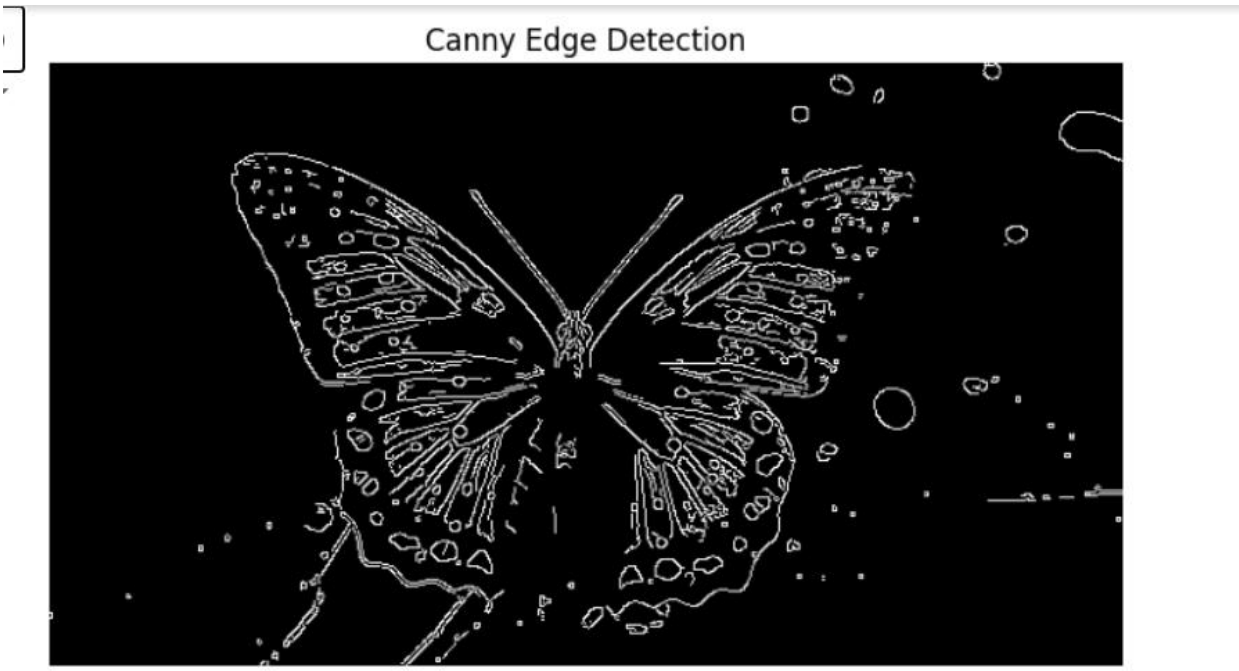
12.1 Detecting the edges

```
# Detecting the edges using Canny Edge Detection
edges = cv2.Canny(gray_image, threshold1=100, threshold2=200)
```

12.2 Visualizing the edges

```
# Detecting the corners using Harris Corner Detection Algorithm
corners = cv2.cornerHarris(np.float32(gray_image), blockSize=2, ksize=3, k=0.04)
```

```
# Visualizing the corners
corner_image = image_rgb.copy()
corner_image[corners > 0.01 * corners.max()] = [255, 0, 0]
plt.figure(figsize=(8, 8))
plt.imshow(corner_image)
plt.title('Corners Detected')
plt.axis('off')
plt.show()
```



13 Corner Detection

12.1 Harris Corner Detection Algorithm

The Harris Corner Detection algorithm is used to identify corner points in an image. Corners are defined as points where the image gradients change significantly in multiple directions. The Harris Corner Detection algorithm identifies corners by calculating the gradient products, applying Gaussian smoothing, computing a corner response function, and then performing thresholding and dilation to highlight the corners on the original image. The following steps outline the algorithm.

1. Convert Image to Grayscale

• Convert the input image to a grayscale image: $I(x,y) = 0.299 \cdot R(x,y) + 0.587 \cdot G(x,y) + 0.114 \cdot B(x,y)$
• Where $R(x,y)$, $G(x,y)$, and $B(x,y)$ are the red, green, and blue color channels, respectively.

2. Compute Image Gradients

• Calculate the gradients of the grayscale image in the x and y directions using filters (e.g., Sobel operators):

$$Ix = \frac{6I}{6x}$$

$$Iy = \frac{6I}{6y}$$

3. Compute Gradient Products

• Compute the products of the gradients:

$$I^2{}_x = Ix \cdot Ix$$

$I^2{}_y = Iy \cdot Iy$

$IxIy = Ix \cdot Iy$

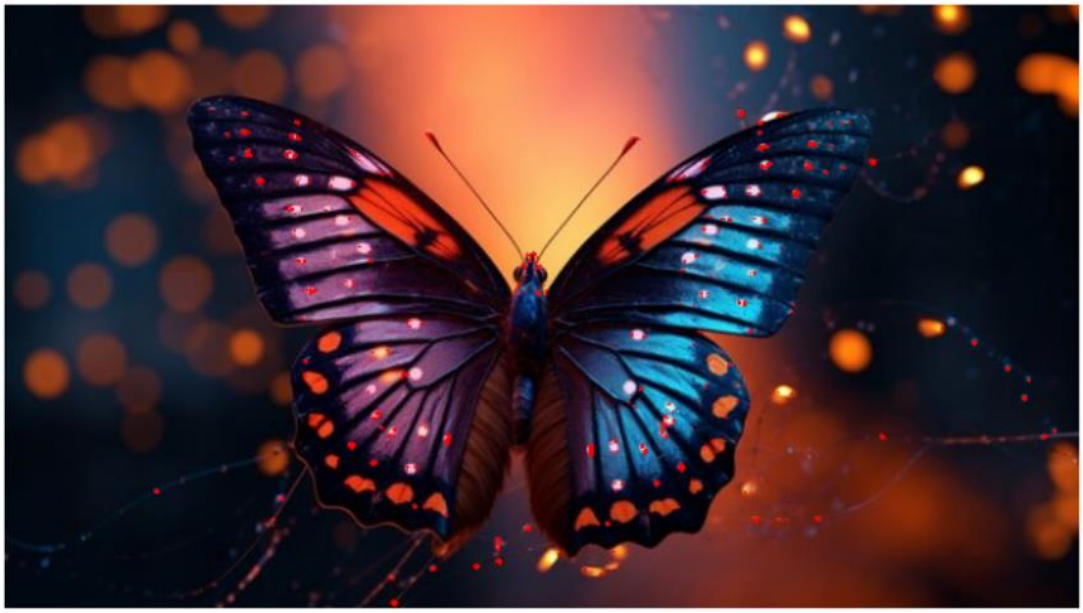
13.1 Detecting the corners using Harris Corner Detection Algorithm

```
# Detecting the corners using Harris Corner Detection Algorithm
corners = cv2.cornerHarris(np.float32(gray_image), blockSize=2, ksize=3, k=0.04)
```

13.2 Visualizing the corners

```
# Visualizing the corners
corner_image = image_rgb.copy()
corner_image[corners > 0.01 * corners.max()] = [255, 0, 0]
plt.figure(figsize=(8, 8))
plt.imshow(corner_image)
plt.title('Corners Detected')
plt.axis('off')
plt.show()
```

Corners Detected



14 More Feature Extraction Techniques

1. **Convolutional Neural Networks (CNNs)**
    - **What they do:** CNNs are powerful tools for analyzing images. They can find detailed patterns and features for tasks like recognizing objects or dividing images into parts.
    - **Why use them:** They work well even if the image changes in lighting, size, or angle. They're very accurate and handle big, complex datasets.
    - **Best for:** Tasks like identifying objects or detecting features with high accuracy.
2. **Autoencoders**
    - **What they do:** Autoencoders learn patterns without needing labeled data. They help in compressing data, cleaning noisy images, or reducing complexity.
    - **Why use them:** Good when CNNs are too expensive or when there's little labeled data.
    - **Best for:** Tasks with messy or large data. Can work with CNNs to improve results.
3. **Histogram of Oriented Gradients (HOG)**
    - **What it does:** HOG looks at the edges and shapes in small parts of the image to identify objects.
    - **Why use it:** Works well with small datasets or blurry images and can be paired with machine learning models like SVM.

- **Best for:** Detecting faces, objects, pedestrians, or text when data is limited.

4. **Scale-Invariant Feature Transform (SIFT)**
    - **What it does:** SIFT finds key points in an image and extracts features that don't change when the image is resized, rotated, or tilted.
    - **Why use it:** It's reliable even if the image changes in size or angle.
    - **Best for:** Tasks like matching objects, creating panoramas, or 3D models with small datasets.
5. **Frequency-Based Features**
    - **What it does:** Uses methods like Fourier Transform to study patterns in the frequency of shapes or textures in an image.
    - **Why use it:** Helps find patterns that don't change with rotation or shifting.
    - **Best for:** Recognizing textures, identifying objects, compressing images, or finding similar images in a database.

    These techniques offer diverse approaches for feature extraction, each with its unique advantages and best-use scenarios. The choice of method depends on the specific problem, available data, and computational resources.